

第 9 集 · 约 22 分钟

请人来攻击： 安全审查与加固

修好了，复验却发现修复本身就是新漏洞

配套阅读：书的第 9 章



这一集你会学到

1. 限速只限速度、不限总量：算一算口令被猜中的概率
2. 先看请求头再读正文：`into_parts`、信号量、`timeout`
3. 孤儿规则：Python 能随手打补丁，Rust 为什么不让
4. 同名不同义：`os.rename` 不覆盖，`fs::rename` 覆盖
5. 防御措施本身也是攻击面；用变异测试证明测试真有用

测试全绿，就安全了吗？

98 个 Rust 测试、22 个 Node 测试全绿，抓包测试也证明网络上看不到秘密。这就安全了吗？

这些测试都是我自己写的。我想到的攻击，我都防了；我没想到的，测试里也不会有。

审查结果：没有 Critical，但是……

编号	问题	审查者的实测
I-1	口令可以被长期在线猜测	从 40 个源地址连发 130 秒，服务端每分钟正好受理 20 次猜测；按这个速度，30 天约 58% 猜中
I-2	没传完的上传永不过期	同一会话并发 100 个 <code>upload_begin</code> 全部成功（上限明明是 64）；之后别的设备再也传不了
I-3	不需要密钥就能撑爆内存	300 个连接各扣住请求体最后一个字节，服务端内存涨了 526 MiB，60 秒后一个都没断
M-1	卸载脚本可能删掉用户文件	在叫 LanShare 的共享文件夹里运行卸载脚本，整个文件夹连同用户文件被永久删除，还提示“文件没有动”
M-2	任何网页都能替你发口令尝试	<code>text/plain</code> 类型的跨站请求直接被受理
M-3	同名文件可能被悄悄覆盖	Rust 的 <code>fs::rename</code> 在 Windows 上覆盖已存在的文件

I-1：限速了，为什么还会被猜中

每分钟猜几次 20

连续猜几天 30

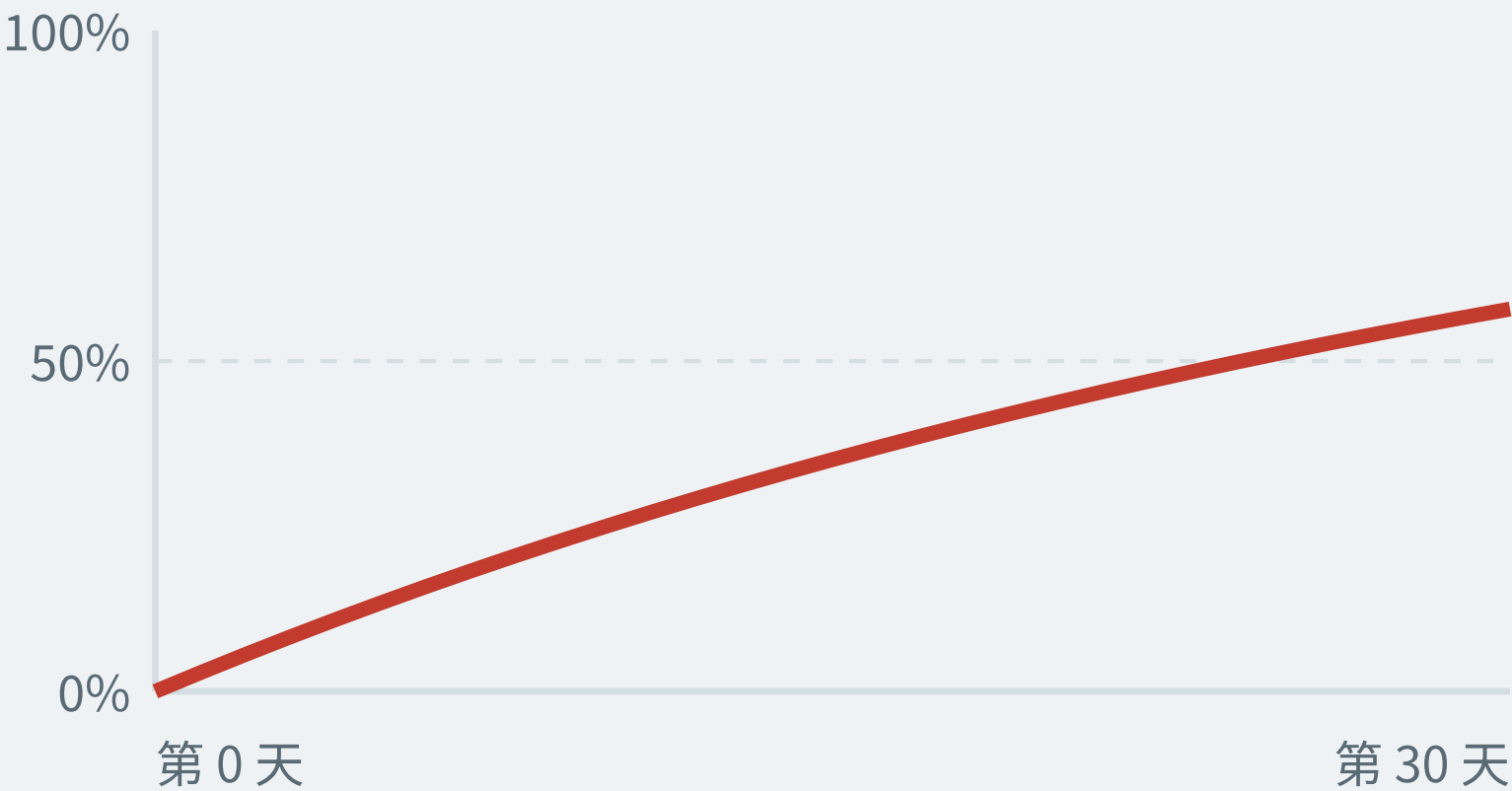
☐ 累计 30 次没登录成功就暂停口令登录（v2 现在的做法）

至少猜中一次的概率

57.85%

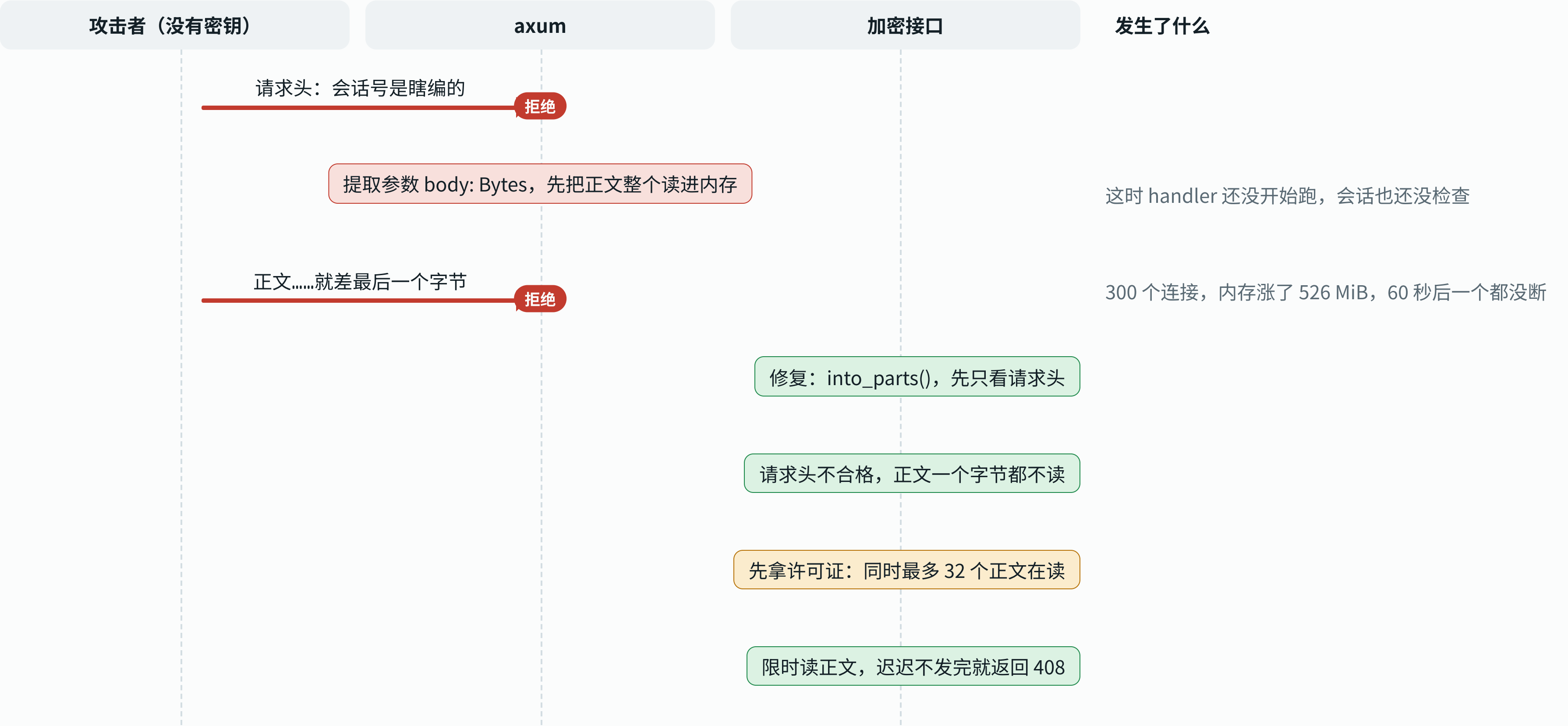
$20 \times 1440 \times 30 = 864,000$ 次 $\rightarrow 1 - (1 - 10^{-6})^{\text{次数}}$

概率随天数变化



6 位数字口令，每次猜中的概率是 $1/1,000,000$ ；SPAKE2 保证每次交互只能验证一个口令。

I-3：没有密钥，也能撑爆内存



先看请求头，再读正文

server/channel.rs

```
1 pub(crate) async fn sealed(state: St, ConnectInfo(peer): ConnectInfo<SocketAddr>, request: Request) -> Response {
2     let st = state.0;
3     let (parts, body) = request.into_parts();           // 拆成“头”和“还没读的正文”
4     let path = parts.uri.path().to_string();
5     let (cid, ctr, keys) = match precheck(&st, &parts.headers, &path) {
6         Ok(checkered) => checked,                       // 类型、长度、会话、计数器都合格
7         Err(status) => return status.into_response(),    // 不合格：正文一个字节都不读
8     };
9     // 先占本 IP 的名额（满了立刻 503），再排全局的队（最多等 30 秒）
10    let Some(_mine) = BodySlot::take(&st, peer.ip()) else {
11        return StatusCode::SERVICE_UNAVAILABLE.into_response();
12    };
13    let Ok(Ok(_permit)) = tokio::time::timeout(QUEUE_WAIT, st.bodies.clone().acquire_owned()).await else {
14        return StatusCode::SERVICE_UNAVAILABLE.into_response();
15    };
16    let body = match tokio::time::timeout(st.body_timeout, axum::body::to_bytes(body, body_limit(&path))).await {
17        Ok(Ok(body)) => body,
18        Ok(Err(_)) => return StatusCode::PAYLOAD_TOO_LARGE.into_response(),
19        Err(_) => return StatusCode::REQUEST_TIMEOUT.into_response(),
20    };
```

孤儿规则：编译器不让打补丁

Windows PowerShell

```
PS D:\lanshare> cargo build
```

```
error[E0117]: only traits defined in the current crate can be implemented for types defined outside of the crate
```

```
--> crates\lanshare\src\server\conn.rs:127:1
```

```
|
```

```
127 | impl axum::extract::connect_info::Connected<axum::serve::IncomingStream<'_, GuardedListener>> for SocketAddr {
```

```
|
```

```
|
```

```
`std::net::SocketAddr` is not defined in the current crate
```

```
|     `IncomingStream` is not defined in the current crate
```

```
= note: define and implement a trait or new type instead
```

同名函数，不同语义

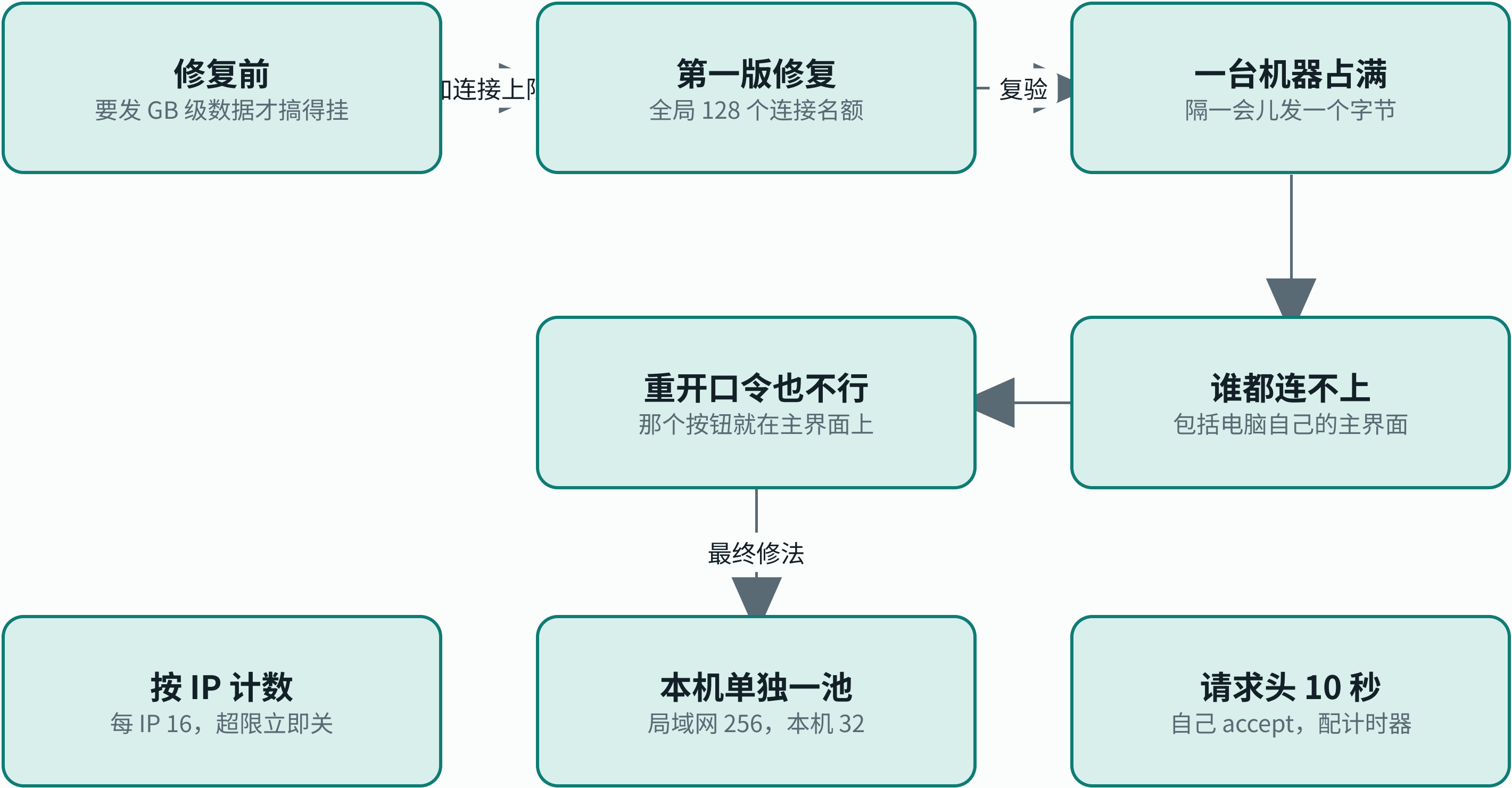
Python v1 · storage.py

```
1 with self._lock:
2     final = self._free_path(safe)
3     os.rename(tmp, final)
```

Rust v2 第一版 · storage.rs

```
1 if target.exists() {
2     continue;
3 }
4 match fs::rename(temp, &target) {
5     Ok(()) => return Ok(candidate),
6     // 别的程序恰好同时建了同名文件: Windows 上 rename 不覆盖, 按
7     Err(e) if e.kind() == io::ErrorKind::AlreadyExists =>
```

复验：修复反而降低了门槛



动手：名额怎么分才公平

☒ 每个 IP 最多 2 个（实际 16 个） ☒ 本机单独一池（实际 32 个）

局域网名额（这里 6 个，实际 256 个）

空

空

空

空

空

空

本机名额（这里 2 个）

空

空

- 捣乱的人再开 3 个连接
- 手机连一下
- 电脑上的主界面连一下
- 全部断开

结果

变异测试：把防线改坏，看谁变红

☒ 用补强后的测试（取消勾选 = 第一次跑的测试）

故意改坏	对应的测试	结果
☐ 空闲计时永不到期	idle_connections_are_closed	✓ 通过
☐ 去掉每会话上传名额	uploads_per_session_are_capped	✓ 通过
☐ 不检查会话就去读正文	unknown_session_is_rejected_before_the_body_arrives	✓ 通过
☐ rename_no_replace 改回 fs::rename	finalize_numbers_duplicates	✓ 通过
☐ 去掉 pake_start 里的“已暂停”检查	pin_login_pauses_after_too_many_unfinished_attempts	✓ 通过
☐ 去掉每 IP 连接上限	connections_per_ip_are_capped	✓ 通过
☐ 去掉请求头时限	headers_must_arrive_in_time	✓ 通过

勾选左边就是把那道防线改坏。测试失败（变红）= 测试抓住了；测试还是通过 = 变异体“活下来了”，说明测试有漏洞。

真实踩坑：测试本身也会骗人

并发漏洞的测试，修复前就通过了

`#[tokio::test]` 默认单线程，20 个请求其实是一个接一个处理的。改成多线程后，修复前稳定复现 8~9 次（上限 6），修复后正好 6 次

变异测试“恢复”后，测试还是失败

`shutil.move` 挪回来的备份，修改时间比上次编译还早，`cargo` 以为没改动，跑的仍是变异体。恢复后用 `os.utime` 更新时间

测试期待 401，实际收到 413

正文长度照抄了 PoC，太长了，还没查会话就被拒。测试要走到你想测的那条路上

加固后下载从 180 MiB/s 掉到 160 左右

去掉怀疑的那一行重测两次，还是 155~163；原来是机器上另外两个程序正忙。只改一处做对照，再下结论

一行多余的代码，清空了整整一章

用 Python 改文档时多写了 `open(P, "w").write(open(P).read())`，先清空、再读到空的。要改就读进来、改、写回去，写完就提交

选择题

原来的设计里有一条“失败 10 次自动换口令”。它能降低攻击者猜中的概率吗？

- A 能，换了口令，攻击者之前排除掉的那些就白猜了
- B 不能，每次猜中的概率还是百万分之一，换口令只会打扰正常用户
- C 不能，但也不需要：SPAKE2 本来就挡住了猜口令
- D 和限速叠在一起就能

选择题

原来的设计里有一条“失败 10 次自动换口令”。它能降低攻击者猜中的概率吗？

A 能，换了口令，攻击者之前排除掉的那些就白猜了

攻击者每次猜的本来就是随机数，不靠排除。口令换不换，每次猜中的概率都是百万分之一。

B 不能，每次猜中的概率还是百万分之一，换口令只会打扰正常用户

对。真正有效的是限制总量：累计 30 次没完成的尝试就暂停，必须有人在电脑前重新开启。

C 不能，但也不需要：SPAKE2 本来就挡住了猜口令

SPAKE2 挡住的是抓包后的离线穷举。在线猜时，每次交互仍然能验证一个口令，只能靠限制次数来防。

D 和限速叠在一起就能

原设计本来就是两者叠加：每分钟 20 次加失败换口令，30 天照样约 58% 被猜中。

选择题

下面两行有什么区别？哪一行会让“同时最多 32 个正文”的限制失效？

```
1 let _ = sem.clone().acquire_owned().await;  
2 let _permit = sem.clone().acquire_owned().await;
```

A 第一行：许可证当场被丢弃，立刻就还回去了

B 第二行：_permit 没被用到，编译器会把它优化掉

C 两行一样，都持有许可证到作用域结束

D 两行都会失效，许可证要手动 release

选择题

下面两行有什么区别？哪一行会让“同时最多 32 个正文”的限制失效？

```
1 let _ = sem.clone().acquire_owned().await;  
2 let _permit = sem.clone().acquire_owned().await;
```

A 第一行：许可证当场被丢弃，立刻就还回去了

对。let _ = 不绑定变量，值当场 drop，许可证马上归还，等于没限制。这是第 7 集 let _ 陷阱的又一次出现。

B 第二行：_permit 没被用到，编译器会把它优化掉

下划线开头只是让编译器不报“没用到”的警告。_permit 照样活到作用域结束，Drop 的时机不会因为优化而改变。

C 两行一样，都持有许可证到作用域结束

不一样。单独的 _ 不是变量名，_permit 是。前者当场丢弃，后者活到大括号结束。

D 两行都会失效，许可证要手动 release

不用手动。持有许可证的变量离开作用域时自动归还，这正是 RAII。

想一想

加固之后，`SECURITY.md` 仍然如实写着：局域网里的人可以让服务变慢或连不上。既然拒绝服务还是防不住，花这么大力气加固图什么？现在到底保证了什么？

想一想

加固之后，`SECURITY.md` 仍然如实写着：局域网里的人可以让服务变慢或连不上。既然拒绝服务还是防不住，花这么大力气加固图什么？现在到底保证了什么？

保证的是：不会因此泄密，不会把内存或磁盘耗尽，本机界面始终有自己的名额（局域网占满了，电脑上的主界面照样能用），扫码登录也不受口令暂停的影响。加固前，几百个不发完的连接就能让内存涨几百 MiB；现在资源占用有上界。至于“完全挡住局域网里的捣乱”，本来就做不到：在局域网里捣乱，本来也可以直接干扰 WiFi。把做不到的事如实写下来，也是安全工作的一部分。

这一集我们学了

- ❌ 自己写的测试只防得住自己想到的攻击，所以要请外人来，每个发现都要 PoC
- ❌ 限速只限速度、不限总量；时间够长，再慢的猜测也会成功
- ❌ 先看请求头再读正文：`into_parts`、信号量、`timeout`；许可证和“票”都靠 Drop 自动归还
- ❌ `os.rename` 不覆盖，`fs::rename` 覆盖：同名函数，语义可能不同
- ❌ 防御措施本身也是攻击面；变异测试证明测试有用，有几道防线就分别测几道

下一集

全课完结，感谢收看